DS H.W SET 5

1. Use linked list for polynomial representation and perform addition & subtraction operations on the polynomials.

Code :

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int coeff;
   int exp;
   struct Node *next;
};
struct Node* createNode(int coeff, int exp)
{
   struct Node *newNode =
      (struct Node*)malloc(sizeof(struct Node));
   newNode->coeff = coeff;
   newNode->exp = exp;
   newNode->next = NULL;
   return newNode;
}
void insertTerm(struct Node **head, int coeff, int exp)
{
   struct Node *newNode = createNode(coeff, exp);
   if(*head == NULL)
   {
      *head = newNode;
      return;
   }
   struct Node *temp = *head;
   while(temp->next != NULL)
   {
      temp = temp->next;
```

```
    }
    temp->next = newNode;
}
void display(struct Node *head)
{
    while(head != NULL)
    {
        printf("%dx^%d", head->coeff, head->exp);
        if(head->next != NULL)
            printf(" + ");
        head = head->next;
    }
    printf("\n");
}
struct Node* addPoly(struct Node *p1, struct Node *p2)
{
    struct Node *result = NULL;
    while(p1 && p2)
    {
        if(p1->exp > p2->exp)
        {
            insertTerm(&result, p1->coeff, p1->exp);
            p1 = p1->next;
        }
        else if(p1->exp < p2->exp)
        {
            insertTerm(&result, p2->coeff, p2->exp);
            p2 = p2->next;
        }
        else
        {
            insertTerm(&result,
                    p1->coeff + p2->coeff,
```

```
                p1->exp);
            p1 = p1->next;
            p2 = p2->next;
        }
    }
    while(p1)
    {
        insertTerm(&result, p1->coeff, p1->exp);
        p1 = p1->next;
    }
    while(p2)
    {
        insertTerm(&result, p2->coeff, p2->exp);
        p2 = p2->next;
    }
    return result;
}
struct Node* subtractPoly(struct Node *p1, struct Node *p2)
{
    struct Node *result = NULL;
    while(p1 && p2)
    {
        if(p1->exp > p2->exp)
        {
            insertTerm(&result, p1->coeff, p1->exp);
            p1 = p1->next;
        }
        else if(p1->exp < p2->exp)
        {
            insertTerm(&result, -p2->coeff, p2->exp);
            p2 = p2->next;
        }
        else
```

```
        {
            insertTerm(&result,
                    p1->coeff - p2->coeff,
                    p1->exp);
            p1 = p1->next;
            p2 = p2->next;
        }
    }
    while(p1)
    {
        insertTerm(&result, p1->coeff, p1->exp);
        p1 = p1->next;
    }
    while(p2)
    {
        insertTerm(&result, -p2->coeff, p2->exp);
        p2 = p2->next;
    }
    return result;
}
int main()
{
    struct Node *poly1 = NULL;
    struct Node *poly2 = NULL;
    struct Node *sum = NULL;
    struct Node *diff = NULL;
    int n, coeff, exp;
    printf("Enter number of terms in Polynomial 1: ");
    scanf("%d", &n);
    printf("Enter coefficient and exponent in descending order:\n");
    for(int i = 0; i < n; i++)
    {
        scanf("%d %d", &coeff, &exp);
```

```
        insertTerm(&poly1, coeff, exp);
    }
    printf("Enter number of terms in Polynomial 2: ");
    scanf("%d", &n);
    printf("Enter coefficient and exponent in descending order:\n");
    for(int i = 0; i < n; i++)
    {
        scanf("%d %d", &coeff, &exp);
        insertTerm(&poly2, coeff, exp);
    }
    printf("\nPolynomial 1:\n");
    display(poly1);
    printf("\nPolynomial 2:\n");
    display(poly2);
    sum = addPoly(poly1, poly2);
    diff = subtractPoly(poly1, poly2);
    printf("\nAddition Result:\n");
    display(sum);
    printf("\nSubtraction Result (P1 - P2):\n");
    display(diff);
    return 0;
}
```

Output

```
Enter number of terms in Polynomial 1: 3
Enter coefficient and exponent in descending order:
5 2
4 1
2 0
Enter number of terms in Polynomial 2: 3
Enter coefficient and exponent in descending order:
3 2
2 1
1 0
```

Polynomial Representation

P1 = $5x^2 + 4x^1 + 2$

P2 = $3x^2 + 2x^1 + 1$

Polynomial 1:

5x^2 + 4x^1 + 2x^0

Polynomial 2:

3x^2 + 2x^1 + 1x^0

Addition Result:

8x^2 + 6x^1 + 3x^0

Subtraction Result (P1 - P2):

2x^2 + 2x^1 + 1x^0

2. Find previous node of circular LL.

Code

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node *next;
};
struct Node *head = NULL;
// Create Circular Linked List
void createList(int n)
{
   struct Node *newNode, *temp;
   for(int i = 1; i <= n; i++)
   {
      newNode = (struct Node *)malloc(sizeof(struct Node));
      printf("Enter data for node %d: ", i);
      scanf("%d", &newNode->data);
      newNode->next = NULL;
      if(head == NULL)
      {
         head = newNode;
         temp = newNode;
      }
      else
      {
         temp->next = newNode;
         temp = newNode;
      }
   }
   temp->next = head; // Make circular
}
```

```
// Display Circular Linked List
void display()
{
   struct Node *temp = head;
   if(head == NULL)
      return;
   printf("\nCircular Linked List: ");
   do
   {
      printf("%d -> ", temp->data);
      temp = temp->next;
   }
   while(temp != head);
   printf("(head)\n");
}
void findPreviousNode(int key)
{
   if(head == NULL)
   {
      printf("List is empty.\n");
      return;
   }
   struct Node *current = head;
   struct Node *previous = NULL;
   do
   {
      previous = current;
```

Output

```
Enter number of nodes: 5
Enter data for node 1: 10
Enter data for node 2: 20
Enter data for node 3: 30
```

Enter data for node 4: 40

Enter data for node 5: 50

Enter node value to find its previous node: 30

Circular Linked List: 10 -> 20 -> 30 -> 40 -> 50 -> (head)

Previous node of 30 is 20

2. Enter node value to find its previous node: 10

Previous node of 10 is 50

3. Insert at a given position of a circular LL.

Code

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node *next;
};
struct Node *head = NULL;
void createList(int n)
{
   struct Node *newNode, *temp;
   for(int i = 1; i <= n; i++)
   {
      newNode = (struct Node*)malloc(sizeof(struct Node));
      printf("Enter data for node %d: ", i);
      scanf("%d", &newNode->data);
      if(head == NULL)
      {
         head = newNode;
         temp = newNode;
         newNode->next = head;
      }
      else
      {
         temp->next = newNode;
         temp = newNode;
         temp->next = head;
      }
   }
}
void display()
```

```
{
    if(head == NULL)
        return;
    struct Node *temp = head;
    printf("\nCircular Linked List:\n");
    do
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while(temp != head);
```

Output

```
Enter number of nodes: 4
Enter data for node 1: 10
Enter data for node 2: 20
Enter data for node 3: 30
Enter data for node 4: 40
Enter data to insert: 25
Enter position: 3
Before Insertion:
Circular Linked List:
10 -> 20 -> 30 -> 40 -> HEAD
After Insertion:
Circular Linked List:
10 -> 20 -> 25 -> 30 -> 40 -> HEAD
```

```
2 . Enter data to insert: 5
Enter position: 1
After Insertion:
Circular Linked List:
5 -> 10 -> 20 -> 30 -> 40 -> HEAD
```

4. Find 2nd largest element in a Queue.

Code

```
#include <stdio.h>
#include <limits.h>
#define MAX 100
int main()
{
   int queue[MAX];
   int n;
   printf("Enter the number of elements in the queue: ");
   scanf("%d", &n);
   if(n < 2)
   {
      printf("At least 2 elements are required.\n");
      return 0;
   }
   printf("Enter the queue elements:\n");
   for(int i = 0; i < n; i++)
   {
      scanf("%d", &queue[i]);
   }
   printf("\nQueue Elements: ");
   for(int i = 0; i < n; i++)
   {
      printf("%d ", queue[i]);
   }
   int largest = INT_MIN;
   int secondLargest = INT_MIN;
   for(int i = 0; i < n; i++)
   {
      if(queue[i] > largest)
      {
         secondLargest = largest;
```

```
            largest = queue[i];
        }
        else if(queue[i] > secondLargest && queue[i] != largest)
        {
            secondLargest = queue[i];
        }
    }
    if(secondLargest == INT_MIN)
    {
        printf("\nNo second largest element exists.\n");
    }
    else
    {
        printf("\nSecond Largest Element = %d\n", secondLargest);
    }
    return 0;
}
```

Output

Enter the number of elements in the queue: 6

Enter the queue elements:

10 25 40 15 35 20

Queue Elements: 10 25 40 15 35 20

Second Largest Element = 35

2 . Enter the number of elements in the queue: 5

Enter the queue elements:

50 30 80 60 70

Queue Elements: 50 30 80 60 70

Second Largest Element = 70

3. Enter the number of elements in the queue: 4

Enter the queue elements:

20 20 20 20

Queue Elements: 20 20 20 20

No second largest element exists.

5. Insert 3rd element and check whether 1st and 3rd elements are same in a LL.

Code

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node *next;
};
struct Node *head = NULL;
// Create Linked List
void createList(int n)
{
   struct Node *newNode, *temp;
   for(int i = 1; i <= n; i++)
   {
      newNode = (struct Node *)malloc(sizeof(struct Node));
      printf("Enter data for node %d: ", i);
      scanf("%d", &newNode->data);
      newNode->next = NULL;
      if(head == NULL)
      {
         head = newNode;
         temp = newNode;
      }
      else
      {
         temp->next = newNode;
         temp = newNode;
      }
   }
}
void display()
```

```
{
    struct Node *temp = head;
    printf("\nLinked List:\n");
    while(temp != NULL)
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
void insertThird(int value)
{
    struct Node *newNode, *temp;
    newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = value;
    if(head == NULL)
    {
        printf("List is empty.\n");
        return;
    }
    temp = head;
    for(int i = 1; i < 2 && temp != NULL; i++)
    {
        temp = temp->next;
    }
    if(temp == NULL)
    {
        printf("Cannot insert at 3rd position.\n");
        return;
    }
    newNode->next = temp->next;
    temp->next = newNode;
}
```

```
void checkFirstAndThird()
{
   struct Node *temp = head;
   if(head == NULL || head->next == NULL || head->next->next == NULL)
   {
      printf("List has less than 3 elements.\n");
      return;
   }
   int first = head->data;
   int third = head->next->next->data;
   printf("\n1st Element = %d", first);
   printf("\n3rd Element = %d\n", third);
   if(first == third)
      printf("1st and 3rd elements are SAME.\n");
   else
      printf("1st and 3rd elements are NOT SAME.\n");
}
int main()
{
   int n, value;
   printf("Enter number of nodes: ");
   scanf("%d", &n);
   createList(n);
   printf("\nBefore Insertion:");
   display();
   printf("\nEnter value to insert at 3rd position: ");
   scanf("%d", &value);
   insertThird(value);
   printf("\nAfter Insertion:");
   display();
   checkFirstAndThird();
   return 0;
}
```

Output

Enter number of nodes: 4

Enter data for node 1: 10

Enter data for node 2: 20

Enter data for node 3: 30

Enter data for node 4: 40

Enter value to insert at 3rd position: 10

Before Insertion:

Linked List:

10 -> 20 -> 30 -> 40 -> NULL

After Insertion:

Linked List:

10 -> 20 -> 10 -> 30 -> 40 -> NULL

1st Element = 10

3rd Element = 10

1st and 3rd elements are SAME.

2. List: 10 -> 20 -> 30 -> 40

Insert: 25

After Insertion:

10 -> 20 -> 25 -> 30 -> 40 -> NULL

1st Element = 10

3rd Element = 25

1st and 3rd elements are NOT SAME.

6. Implement two stacks in one array. Write Routine to traverse the array and display.

```
#include <stdio.h>
#define MAX 10
int arr[MAX];
int top1 = -1;
int top2 = MAX;
void push1(int value)
{
   if(top1 + 1 == top2)
   {
      printf("Stack Overflow!\n");
      return;
   }
   arr[++top1] = value;
}
void push2(int value)
{
   if(top1 + 1 == top2)
   {
      printf("Stack Overflow!\n");
      return;
   }
   arr[--top2] = value;
}
void displayStack1()
{
   if(top1 == -1)
   {
      printf("Stack 1 is Empty\n");
      return;
   }
   printf("\nStack 1 Elements:\n");
   for(int i = top1; i >= 0; i--)
```

```
    {
        printf("%d\n", arr[i]);
    }
}
void displayStack2()
{
    if(top2 == MAX)
    {
        printf("Stack 2 is Empty\n");
        return;
    }
    printf("\nStack 2 Elements:\n");
    for(int i = top2; i < MAX; i++)
    {
        printf("%d\n", arr[i]);
    }
}
void displayArray()
{
    printf("\nComplete Array:\n");
    for(int i = 0; i < MAX; i++)
    {
        printf("arr[%d] = %d\n", i, arr[i]);
    }
}
int main()
{
    int n1, n2, value;
    // Initialize array
    for(int i = 0; i < MAX; i++)
        arr[i] = -1;
    printf("Enter number of elements for Stack 1: ");
```

Output

Enter number of elements for Stack 1: 4

Enter elements for Stack 1:

10

20

30

40

Enter number of elements for Stack 2: 3

Enter elements for Stack 2:

50

60

70

Stack 1 Elements:

40

30

20

10

Stack 2 Elements:

70

60

50

Complete Array:

arr[0] = 10

arr[1] = 20

arr[2] = 30

arr[3] = 40

arr[4] = -1

arr[5] = -1

arr[6] = -1

arr[7] = 70

arr[8] = 60

arr[9] = 50

7. Given a linked list containing n head nodes where every node in the LL contains two pointers:

- next points to next node in the LL

- bottom pointer to a sub linked list where the current node is the head. Each of the sub-linked list's nodes and the head nodes are sorted in ascending order based on their data. We need to flatten the LL so that all the nodes appear in a single LL while maintaining sorted order.

Code

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node *next;
   struct Node *bottom;
};
struct Node* createNode(int data)
{
   struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->next = NULL;
   newNode->bottom = NULL;
   return newNode;
}
struct Node* merge(struct Node *a, struct Node *b)
{
   if(a == NULL)
      return b;
   if(b == NULL)
      return a;
   struct Node *result;
   if(a->data < b->data)
   {
      result = a;
      result->bottom = merge(a->bottom, b);
```

```
    }
    else
    {
        result = b;
        result->bottom = merge(a, b->bottom);
    }
    result->next = NULL;
    return result;
}
struct Node* flatten(struct Node *root)
{
    if(root == NULL || root->next == NULL)
        return root;
    root->next = flatten(root->next);
    root = merge(root, root->next);
    return root;
}
void display(struct Node *head)
{
    printf("\nFlattened Linked List:\n");
    while(head != NULL)
    {
        printf("%d ", head->data);
        head = head->bottom;
    }
    printf("\n");
}
int main()
{
    int n;
    printf("Enter number of head nodes: ");
    scanf("%d", &n);
    struct Node *head = NULL;
```

```
struct Node *lastHead = NULL;
for(int i = 1; i <= n; i++)
{
   int count;
   printf("\nEnter number of nodes in sublist %d: ", i);
   scanf("%d", &count);
   struct Node *topNode = NULL;
   struct Node *bottomLast = NULL;
   printf("Enter %d sorted values:\n", count);
   for(int j = 0; j < count; j++)
   {
      int value;
      scanf("%d", &value);
      struct Node *newNode = createNode(value);
      if(topNode == NULL)
      {
         topNode = newNode;
         bottomLast = newNode;
      }
      else
      {
         bottomLast->bottom = newNode;
         bottomLast = newNode;
      }
   }
   if(head == NULL)
   {
      head = topNode;
      lastHead = topNode;
   }
   else
   {
      lastHead->next = topNode;
```

```
            lastHead = topNode;
        }
    }
    head = flatten(head);
    display(head);
    return 0;
}
```

Output

```
Enter number of head nodes: 4
Enter number of nodes in sublist 1: 3
Enter 3 sorted values:
5 7 8
Enter number of nodes in sublist 2: 2
Enter 2 sorted values:
10 20
Enter number of nodes in sublist 3: 2
Enter 2 sorted values:
19 22
Enter number of nodes in sublist 4: 3
Enter 3 sorted values:
28 40 45
Flattened Linked List:
5 7 8 10 19 20 22 28 40 45
```

8. Given k sorted LL of different sizes, we need to merge them into a single list maintaining their sorted order.

```
#include <stdio.h>
#include <stdlib.h>
// Node structure
struct Node
{
   int data;
   struct Node *next;
};
struct Node* createNode(int data)
{
   struct Node *newNode =
      (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->next = NULL;
   return newNode;
}
void insert(struct Node **head, int data)
{
   struct Node *newNode = createNode(data);
   if(*head == NULL)
   {
      *head = newNode;
      return;
   }
   struct Node *temp = *head;
   while(temp->next != NULL)
      temp = temp->next;
   temp->next = newNode;
}
struct Node* mergeTwoLists(struct Node *a, struct Node *b)
{
   if(a == NULL)
```

```
        return b;
    if(b == NULL)
        return a;
    struct Node *result = NULL;
    if(a->data <= b->data)
    {
        result = a;
        result->next = mergeTwoLists(a->next, b);
    }
    else
    {
        result = b;
        result->next = mergeTwoLists(a, b->next);
    }
    return result;
}
struct Node* mergeKLists(struct Node *lists[], int k)
{
    struct Node *result = NULL;
    for(int i = 0; i < k; i++)
    {
        result = mergeTwoLists(result, lists[i]);
    }
    return result;
}
```

Output

```
Enter number of linked lists (K): 3
Enter number of nodes in List 1: 4
Enter 4 sorted elements:
1 4 7 10
Enter number of nodes in List 2: 3
Enter 3 sorted elements:
2 5 8
```

Enter number of nodes in List 3: 5

Enter 5 sorted elements:

3 6 9 11 12

Merged Sorted Linked List:

1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 -> 10 -> 11 -> 12 -> NULL

2. K = 4

List 1: 1 3 5

List 2: 2 4 6

List 3: 7 8

List 4: 9 10 11

Merged Sorted Linked List:

1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 -> 10 -> 11 -> NULL

9. Swap k-th node from beginning with k-th node from end in a LL.

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node *next;
};
struct Node* createNode(int data)
{
   struct Node *newNode =
      (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->next = NULL;
   return newNode;
}
void insert(struct Node **head, int data)
{
   struct Node *newNode = createNode(data);
   if(*head == NULL)
   {
      *head = newNode;
      return;
   }
   struct Node *temp = *head;
   while(temp->next != NULL)
      temp = temp->next;
   temp->next = newNode;
}
void display(struct Node *head)
{
   while(head != NULL)
   {
```

```
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}
int countNodes(struct Node *head)
{
    int count = 0;
    while(head != NULL)
    {
        count++;
        head = head->next;
    }
    return count;
}
void swapKthNode(struct Node *head, int k)
{
    int n = countNodes(head);
    if(k > n)
    {
        printf("\nInvalid value of k.\n");
        return;
    }
    if(2 * k - 1 == n)
    {
        printf("\nBoth nodes are same. No swap needed.\n");
        return;
    }
    struct Node *first = head;
    struct Node *second = head;
```

Output

```
Enter number of nodes: 5
Enter node values:
```

10 20 30 40 50

Enter value of k: 2

10 -> 20 -> 30 -> 40 -> 50

Linked List after swapping:

10 -> 40 -> 30 -> 20 -> 50 -> NULL

2. Enter number of nodes: 7

Enter node values:

1 2 3 4 5 6 7

Enter value of k: 3

1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7

Linked List after swapping:

1 -> 2 -> 5 -> 4 -> 3 -> 6 -> 7 -> NULL

3. Enter number of nodes: 5

Enter node values:

10 20 30 40 50

Enter value of k: 3

Both nodes are same. No swap needed.

Linked List after swapping:

10 -> 20 -> 30 -> 40 -> 50 -> NULL

10. Multiply two numbers represented by two LL.

eg. [1] -> [0] -> [0]

X [1] -> [0]

= 1 -> 0 -> 0 -> 0

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node *next;
};
struct Node* createNode(int data)
{
   struct Node* newNode =
      (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->next = NULL;
   return newNode;
}
void insert(struct Node **head, int data)
{
   struct Node *newNode = createNode(data);
   if(*head == NULL)
   {
      *head = newNode;
      return;
   }
   struct Node *temp = *head;
   while(temp->next != NULL)
      temp = temp->next;
   temp->next = newNode;
}
void display(struct Node *head)
```

```
{
    while(head != NULL)
    {
        printf("%d", head->data);
        if(head->next != NULL)
            printf(" -> ");
        head = head->next;
    }
    printf("\n");
}
long long listToNumber(struct Node *head)
{
    long long num = 0;
    while(head != NULL)
    {
        num = num * 10 + head->data;
        head = head->next;
    }
    return num;
}
struct Node* numberToList(long long num)
{
    struct Node *head = NULL;
    if(num == 0)
    {
        insert(&head, 0);
        return head;
    }
    int digits[50];
    int count = 0;
    while(num > 0)
    {
        digits[count++] = num % 10;
```

```
        num /= 10;
    }
    for(int i = count - 1; i >= 0; i--)
    {
        insert(&head, digits[i]);
    }
    return head;
}
int main()
{
    struct Node *num1 = NULL;
    struct Node *num2 = NULL;
    struct Node *result = NULL;
    int n1, n2, digit;
    printf("Enter number of digits in First Number: ");
    scanf("%d", &n1);
    printf("Enter digits of First Number:\n");
    for(int i = 0; i < n1; i++)
    {
        scanf("%d", &digit);
        insert(&num1, digit);
    }
    printf("Enter number of digits in Second Number: ");
    scanf("%d", &n2);
    printf("Enter digits of Second Number:\n");
    for(int i = 0; i < n2; i++)
    {
        scanf("%d", &digit);
        insert(&num2, digit);
    }
    printf("\nFirst Number : ");
    display(num1);
    printf("Second Number: ");
```

```
    display(num2);
    long long number1 = listToNumber(num1);
    long long number2 = listToNumber(num2);
    long long product = number1 * number2;
    result = numberToList(product);
    printf("\nProduct Linked List:\n");
    display(result);
    return 0;
}
```

Output

```
Enter number of digits in First Number: 3
Enter digits of First Number:
1 0 0
Enter number of digits in Second Number: 2
Enter digits of Second Number:
1 0
First Number : 1 -> 0 -> 0
Second Number: 1 -> 0
Product Linked List:
1 -> 0 -> 0 -> 0
2. Enter number of digits in First Number: 3
Enter digits:
1 2 3
Enter number of digits in Second Number: 2
Enter digits:
4 5
First Number : 1 -> 2 -> 3
Second Number: 4 -> 5
Product Linked List:
5 -> 5 -> 3 -> 5
```